Vetwise Clinic - Technical Handover

Complete project walkthrough for the course handover.

Prepared: 12 October 2026

Scope: repository structure, Oracle Database Free scripts, Node/Express demonstration website, browser UI, identity, persisted demo data, tests, limitations and operating procedures.

Contents and reading route

Pages 1-8 orient the reader to the project, its parts and data model. Pages 9-21 explain Oracle tables, data, queries, views, PL/SQL, triggers, security and tests. Pages 22-39 explain the web interface, server, routes, login, data, design and testing. Pages 40-48 cover documentation, operation, handover, known limits and the complete file inventory.

Read the Oracle and web sections independently: they are separate implementations of clinic concepts. The browser site is not a visual client of Oracle. The last pages provide runbooks and a demonstration path.

The explanations describe each file's major code blocks, inputs, outputs, state changes and dependencies. For exact spelling and full syntax, use the linked source files in the repository.

1. Project at a glance

This repository is a teaching project with two related but independent deliverables.

1. sql/ rebuilds a relational veterinary-clinic schema in Oracle Database Free. It demonstrates
Oracle DDL, sample DML, 18 reporting queries, views, a PL/SQL package, triggers, grants, transactions and database tests.
2. web/ serves a local fictional staff website using Node.js and Express. It demonstrates
sign-in, administrator/veterinarian roles, an appointment workflow, patient search, invoice display and browser rendering.

The web demo stores its own fictional data in local JSON files. It currently does not use the Oracle database, the oracledb package, a SQL connection, or shared data between the two parts. Both parts model a clinic, but they do not synchronize.

2. Technology and system boundary

The database deliverable targets Oracle Database Free and is executed with SQLcl. The web deliverable uses Node.js, Express 5 and browser-native HTML/CSS/JavaScript. The tests use Node's built-in test runner and the fetch API. No build tool or frontend framework is required.

Boundary map:

* SQLcl sends Oracle-specific SQL and PL/SQL to an Oracle schema.
* Node listens on a local HTTP port, serves static browser files, and answers JSON API routes.
* Browser JavaScript calls the JSON API on the same server and turns returned records into UI.
* Web demo JSON storage and account hashes live only on the computer that runs the server.

An Oracle integration would require server-side connection configuration and changed routes. It is future work, not an existing feature. No database credentials belong in browser code.

3. Whole-repository map

At the project root: README.md is the project guide; AGENTS.md records project rules; .gitignore excludes local secrets, Oracle exports and logs; .DS_Store is a macOS Finder metadata file, not application logic.

sql/ contains ten ordered Oracle scripts. docs/ contains requirements, database design, the ER diagram, normalization, decisions, checklist, backup guide, this handover and user guide. web/ contains its own package manifest, lock file, local environment example and ignore rules, server, route modules, demo data, static interface, DOM checker and API tests.

The layout separates executable database scripts from the web server and separates public static files from server-side route/data modules. Run SQL from the repository root; run the website from the web/ folder.

4. Configuration, ignored files and data safety

Root .gitignore excludes Oracle Data Pump .dmp files, .log files, .env, and the local account and clinic-store JSON files (plus their temporary atomic-write files). web/.gitignore excludes node_modules/ and .env. These exclusions are intentional: operational credentials and local state must not enter Git.

web/.env.example documents the local port and warns that future Oracle settings stay server-side. It is a template, not a secret file. Never paste real invitation codes or passwords into a tracked document, PDF or source file.

The test suite generates temporary account and clinic stores under web/test/, uses random passwords and invitation codes, sets restrictive permissions, and deletes those files afterward.

5. How to run each deliverable

Website from a terminal:

```
cd web
npm install
npm start
```

Then open http://localhost:3000. npm run dev uses Node's watch mode. PORT can change the listener port through a private environment setting. Stop the process with Ctrl+C.

Database: connect with SQLcl to a suitable Oracle schema, change the SQLcl working directory to the repository root, then run @sql/run_all.sql. This is destructive to the named project schema objects: the first script drops those objects before rebuilding them. Read the reset warning and backup instructions before running it on any schema with data worth preserving.

Neither workflow requires starting the other.

6. Web request life cycle

1. Express serves public/index.html, CSS and JavaScript from web/server.js.
2. The browser loads public/js/main.js as an ES module.
3. On startup, it asks for registration options and the current session.
4. After sign-in, browser requests fetch dashboard, patient, veterinarian, appointment and
administrator-only invoice records from /api/....
5. Express routes authenticate the cookie, enforce the role and scope, validate data, update
in-memory arrays loaded from the local JSON store, and persist mutations.
6. JSON results return to the browser; render functions refresh the visible list and summary.

The API and static site share one origin, so browser fetch sends the same-site session cookie. The SQL schema is outside this path.

7. Persistent state and identity links

web/data/clinic.js defines the fictional practice and initial arrays. On server startup it overlays persisted arrays from clinic-store.json if that local store exists. Writes update the arrays and replace a temporary JSON file by rename.

staff-accounts.json is a separate ignored account store. It holds random salts and scrypt derived hashes, never plaintext passwords. Each veterinarian account is matched to one veterinarian profile, and that profile has a stable veterinarianId. API ownership checks use that ID, not a name typed by the browser.

clinic.js and clinic-store.json are mutable web data. Oracle tables are a different data store with their own identities and constraints.

8. Oracle relational model and relationships

The schema has fourteen tables: OWNERS, SPECIES, BREEDS, PETS, VETERINARIANS, APPOINTMENTS, SERVICES, APPOINTMENT_SERVICES, MEDICATIONS, PRESCRIPTIONS, VACCINATIONS, INVOICES, PAYMENTS and APPOINTMENT_AUDIT.

An owner has many pets; species has many breeds; a breed classifies pets; pets and veterinarians each participate in many appointments. APPOINTMENT_SERVICES joins visits to services, while PRESCRIPTIONS joins appointments to medications. Pets receive vaccinations. An appointment has zero or one invoice; an invoice can receive many payments. Appointment changes create audit records.

All tables have identity primary keys. Foreign keys protect relationships, and checks, unique constraints and not-null rules guard values that must be valid.

9. Normalization and pricing decisions

The design follows 1NF by storing one value per column; 2NF by putting facts in tables where they depend on the record; and 3NF by not copying facts that belong to another entity. docs/normalization.md explains each table individually.

Owners are not copied onto pets' appointment rows in Oracle: relationships use IDs and queries join to names. The browser demo does copy display names into denormalized appointment objects for simple rendering; this is a deliberate demo trade-off, not the SQL design.

Service and medication catalogues hold current prices. Appointment service and prescription lines snapshot the charged unit price. An old invoice therefore does not silently change when the catalogue is edited. Invoice totals are calculated from line items and payments, not copied as a possibly stale total.

10. DDL: identity and reference tables

sql/01_ddl.sql begins with SET DEFINE OFF, then creates tables in parent-before-child order. The OWNERS table stores contact details and creation time. SPECIES stores unique animal types. BREEDS belongs to a species; the combined species/breed name is unique.

VETERINARIANS stores identity, licence, contacts and active state. SERVICES stores unique service names, catalogue prices and active state. MEDICATIONS stores names, inventory quantity, unit price and active state.

GENERATED ALWAYS AS IDENTITY lets Oracle generate IDs and disallows client-supplied identity values. NOT NULL, UNIQUE, CHECK and DEFAULT constraints make data integrity a database rule rather than a UI convention.

11. DDL: clinic and finance tables

The second DDL group defines PETS with owner and breed foreign keys and checks for sex and active flag. APPOINTMENTS links a pet and vet, stores date, status, reason and clinical notes, and limits status to the three Oracle values SCHEDULED, COMPLETED and CANCELLED.

APPOINTMENT_SERVICES stores quantity and snapshot price and prevents the same service from being entered twice for one appointment. PRESCRIPTIONS stores dosage, frequency, duration, quantity and charged price. Both are junction/detail tables.

VACCINATIONS stores pet, vaccine, administration and due dates. INVOICES has a unique appointment reference, so one visit has at most one invoice. PAYMENTS records one positive payment and its method. APPOINTMENT_AUDIT keeps before/after values and actor/time for changes.

12. DDL: indexes and deployment details

The remaining statements create indexes on foreign-key columns and frequently searched dates, owner surname and vaccination due date. Oracle does not automatically add every foreign-key index. Indexes help joins and deletes/updates but cost storage and write work; the indexes here match the expected clinic reports and references.

Primary-key and unique constraints create their own supporting indexes. The explicit indexes cover references and common search predicates such as appointment date.

The DDL uses Oracle-supported VARCHAR2, NUMBER, DATE and identity columns. No data types or auto-increment syntax from PostgreSQL, MySQL or SQLite should be substituted. The script ends with a SQLcl PROMPT so the operator knows that schema objects were created.

13. Sample data script

sql/02_data.sql supplies fictional data only. It inserts parent rows before child rows so foreign keys can be satisfied. Identity columns are omitted and generated by Oracle. Parent rows use one insert per statement: an earlier issue showed that one Oracle multi-table INSERT ALL could reuse identity evaluation and collide on owner keys.

PL/SQL blocks create repeating pets and appointment rows, keeping the sample set compact and repeatable while filling six months of visits. Additional blocks assign service lines, prescriptions, invoices and payments to completed visits. The data deliberately includes unpaid/partial balances and vaccines due soon, allowing reports to return meaningful cases.

The script commits seed data and prints a completion prompt. The sample is a dataset, not an application login system.

14. Reporting queries Q1-Q5

sql/03_queries.sql contains eighteen individually commented business questions.

* Q1 joins completed visits to pet, owner and veterinarian to produce readable history.
* Q2 uses a left join so pets with no visit still appear with a zero count.
* Q3 combines performed service and prescribed medicine detail for completed visits.
* Q4 groups completed visits by veterinarian and uses HAVING to retain workload over five.
* Q5 derives unpaid balance from service, prescription and payment subtotals.

Q5 pre-aggregates each many-row source before joining to invoices. This avoids multiplying service lines by prescription lines by payment rows and thereby overstating the balance. NVL turns a missing subtotal into zero; VOID invoices and non-positive balances are excluded.

15. Reporting queries Q6-Q10

* Q6 compares each pet's appointment count with a scalar average over per-pet counts. It uses a nested subquery and HAVING.
* Q7 uses NOT EXISTS to find active services never used in a service line.
* Q8 filters scheduled appointments in the next 30 days.
* Q9 ranks veterinarians by completed visits with an analytic ranking function.
* Q10 builds a running owner payment total using an analytic SUM() OVER ordered by payment.

Together these examples demonstrate subqueries, anti-joins, date filtering, grouping, analytic functions and ranking. They also show how the schema's foreign keys let reports travel from payment to invoice to appointment to pet to owner.

16. Reporting queries Q11-Q18

* Q11 returns the five highest-revenue services using FETCH FIRST.
* Q12 compares medication inventory to an average.
* Q13 finds vaccines due within 30 days.
* Q14 shows invoice payment totals, retaining invoices with no payment.
* Q15 selects each visited pet's latest completed appointment.
* Q16 finds owners with at least one soon-due vaccine.
* Q17 finds scheduled visits missing an invoice.
* Q18 groups recent completed visits by species over 90 days.

The remaining examples exercise outer joins, existence tests, analytic/latest-row logic, aggregation and top-N output. Each query comment states its clinic question; run the file in SQLcl to inspect result sets against the seed data.

17. Reporting views

sql/04_views.sql creates four reusable views:

* VW_UPCOMING_APPOINTMENTS joins future scheduled visits to pet, owner and veterinarian.
* VW_OWNER_OUTSTANDING_BALANCES rolls service, prescription and payment subtotals into owner
balances without join fan-out.
* VW_VETERINARIAN_WORKLOAD groups completed appointments by vet and calendar month.
* VW_VACCINATIONS_DUE_SOON exposes due records in the next 30-day window.

A view is a named query, not a copied table. It centralizes a useful projection and can simplify reports. It does not automatically store a historical snapshot. These views use SQL definitions appropriate to Oracle and depend on the tables, constraints and seeded records already created.

18. PL/SQL package: public contract and booking

sql/05_package.sql creates package specification VET_CLINIC_PKG and its package body. The specification is the callable interface: BOOK_APPOINTMENT, CALCULATE_INVOICE_TOTAL, REGISTER_VACCINATION and VACCINATION_REMINDERS.

BOOK_APPOINTMENT validates required IDs, date and reason; checks that the pet exists and the vet is active; rejects an existing scheduled appointment for that veterinarian/date; then inserts a scheduled visit. Each invalid input has a specific RAISE_APPLICATION_ERROR code. A successful call prints a DBMS_OUTPUT message.

Important limitation: the sample procedure conflict check compares exact date values rather than implementing the web demo's 30-minute interval model. Oracle DATE also includes time. Do not claim those two implementations enforce identical rules.

19. PL/SQL package: billing and vaccinations

CALCULATE_INVOICE_TOTAL reads the invoice's appointment, sums service line quantity times snapshot price and prescription quantity times snapshot price, and returns both subtotals combined. NVL makes an empty category contribute zero. Missing invoices are converted to a clear custom Oracle error.

REGISTER_VACCINATION validates a pet, vaccine name and non-future administration date, then adds a vaccination with a next due date one year after the administration date. TRUNC compares whole calendar days because Oracle SYSDATE carries a time-of-day.

VACCINATION_REMINDERS validates its day range, opens an explicit parameterized cursor, fetches rows in due-date order, prints one reminder at a time, and closes the cursor. Its exception handler closes an open cursor and re-raises the original error.

20. Oracle triggers

sql/06_triggers.sql defines three triggers. TRG_APPOINTMENT_AUDIT runs after insert or updates to selected appointment fields. Oracle's INSERTING condition distinguishes a new row from a change; :OLD and :NEW values are recorded with USER and SYSDATE.

TRG_REDUCE_MEDICATION_STOCK runs before prescription insert. It rejects a missing or non-positive quantity, then subtracts stock only when sufficient quantity exists. If no row was updated, it distinguishes an unknown medicine from insufficient stock and raises a useful application error. Its predicate makes negative inventory impossible through this operation.

TRG_SET_INVOICE_DATE supplies SYSDATE only if an incoming invoice date is null. The file checks compiler errors and runs trigger tests with savepoints/rollback so test mutations do not remain in seed data.

21. Oracle security and transactions

sql/07_security.sql demonstrates the receptionist, vet and admin database roles and least-privilege table grants. It first checks whether the connected schema has CREATE ROLE; without it, the script reports a notice and lets the complete rebuild continue.

Oracle does not permit a role and user with the same name. If the schema owner is VET, a role named vet conflicts. The script reports that limitation rather than claiming that the role exists. A DBA can prepare another schema owner and grant role-creation rights if the course requires real database roles.

The script demonstrates transaction control with SAVEPOINT, ROLLBACK and COMMIT. It does not create application identities or passwords. Granting a role to a real user is an operator/ DBA action outside this student schema.

22. Oracle test suite and rebuild order

sql/08_tests.sql contains sixteen numbered automated checks for invalid foreign keys, unique and check constraints, package success and failure, invoice totals, vaccination dates/reminders, audit behavior, stock changes and invoice-date defaulting. PL/SQL prints PASS or FAIL.

sql/run_all.sql is the single orchestration entry point. It stops on SQLERROR, disables substitution prompts, and executes in dependency order:

```
drop -> DDL -> sample data -> queries -> views -> package -> triggers
     -> security -> final tests
```

The queries display example output during the rebuild. The final test stage should be checked for FAIL messages. SQLcl must be connected to the intended schema and show DBMS_OUTPUT.

23. Browser page: index.html

web/public/index.html is the static structure and accessible labels for the single-page console. It contains the authentication screen; navigation sidebar; top bar; dashboard summary, seven-day admin agenda, worklist and clinic facts; patient table and search; administrator appointment table/filter; billing table/filter; booking and patient dialogs; and toast/error regions.

Role-specific sections carry admin-only and start hidden. main.js exposes controls only to the correct role, but the server remains the actual permission boundary. IDs are the stable selectors used by the JavaScript; a DOM checker catches missing IDs/classes.

The <script type="module"> loads js/main.js. Static HTML does not contain operational data and does not make direct Oracle queries.

24. Browser logic: state and safe rendering

web/public/js/main.js keeps one state object for the current user, dashboard response, patients, vets, appointments, invoices and selected date. request() uses fetch, parses JSON, turns non-2xx responses into errors, and sends expired sessions back to sign-in.

Small helpers centralize JSON request setup, text escaping, CSS class names, date display, local-date construction and money formatting. Values inserted into HTML are passed through escapeHtml; plain labels are set with textContent. This prevents patient/reason strings from being interpreted as active markup.

Each render... function builds only its own display area from API results. The interface does not treat browser memory as a permanent source of truth; successful mutations trigger an API refresh.

25. Startup, sessions and role-based views

start() writes today's label, fetches /api/auth/options and /api/auth/me in parallel, and either restores the signed-in user and loads clinic data or displays the login page. loadClinicData requests dashboard/patient/vet records and the role-appropriate schedule/invoices before showing the application.

renderProfile() fills the user avatar and title, hides administrator-only controls for doctors, sets the role-specific schedule heading, and renders records. showView() changes the visible section and navigation state. It redirects a non-admin attempt to open appointments/billing back to dashboard.

The delegated document click handler dispatches navigation, date selection, appointment actions and invoice actions. One listener works for buttons inserted later by rendering.

26. Dashboard behavior and the long-range fix

renderDashboard() displays role-specific stat cards, derives future active visits from the logged-in user's already-scoped API list, sorts by date/time, and displays dates/statuses. For administrators, the seven-day agenda is retained: selecting a day filters the compact clinic overview. For veterinarians, the date selector is hidden and every future assigned appointment is shown as one chronological worklist.

The earlier bug was that the veterinarian UI reused the administrator's seven-day date selection. A pending visit weeks away existed on the server but was invisible on the doctor dashboard. The current UI does not date-limit the doctor worklist; each entry explicitly shows its date, status and, if pending, Confirm visit.

The dashboard and API summary remain role-scoped. No veterinarian receives another doctor's appointments.

27. Booking dialog and validation feedback

openBookingDialog() resets the form, fills patient and linked-veterinarian selectors from current API state, sets date and time input limits, opens the native dialog and focuses the first field. submitBooking() reads the selected values, converts the patient/vet IDs to numbers, trims the reason and sends JSON to POST /api/appointments.

On success, it closes the dialog, refreshes the server-backed data, and announces the booked date/time. If the server returns a validation/conflict response, the error is shown within the dialog so entered values can be corrected. Network/server errors use the shared alert handler.

HTML input limits improve usability; API validation is authoritative. The server checks real calendar dates and real time values rather than trusting min/max attributes.

28. Patient, appointment and invoice rendering

renderPatients() creates the patient rows; filterPatients() searches all displayed patient properties case-insensitively. The input event re-renders results immediately.

renderAppointments() prints date, time, pet, owner, veterinarian, reason and status. Only an administrator receives action cells: active records can be cancelled, and elapsed confirmed or legacy scheduled records can be completed. The API rechecks every transition.

renderInvoices() prints invoice amounts/status and administrator-only Mark paid buttons. filterRows() filters status values. renderInvoices() does not call the API itself; mutation and refresh functions control it. Empty-state rows describe when no records match.

29. Express application: server.js

web/server.js loads Express, path and dotenv; imports the API and auth routers; creates the app; and chooses PORT or 3000. express.json() parses request bodies, and express.static() exposes only the public/ directory for browser assets.

The auth router is mounted at /api/auth; the authenticated clinic router is mounted at /api. An explicit JSON 404 for unknown API paths appears before the catch-all route that serves index.html for client-side pages. This order prevents a misspelled API URL from appearing to succeed with HTML.

The final error middleware recognizes invalid JSON and otherwise logs server-side detail while returning a generic 500 response. The module listens only when run directly; when tests require it, they can create an ephemeral test server.

30. Authentication bootstrap and account/profile linking

web/routes/auth.js creates the auth router and reads an ignored account-store path, or an override used by tests. It loads private invitation codes from environment variables; a role's registration option is enabled only when its code is sufficiently long.

At startup, existing veterinarian accounts are linked to profiles by account ID/email, then by a canonicalized name for older accounts. If no profile matches, the server creates a profile and persists it. The account receives the profile's stable veterinarian ID. getVeterinarianRoster() returns only profiles linked to a veterinarian login, preventing the appointment picker from offering a doctor who cannot sign in.

publicAccount() deliberately omits salts and password hashes.

31. Authentication, password and session blocks

Registration validates the name, email, role, specialty and password length, then checks the role-specific invitation. Email uniqueness is case-insensitive. Passwords are derived with Node scrypt, a random per-account salt and explicit cost parameters. Only salt and derived hash are saved, using a temporary file, atomic rename and owner-only file permissions.

Login rate-limits attempts by request address/path, uses a dummy salt for unknown email to reduce timing differences, derives the candidate hash and compares with a timing-safe equality helper. Successful login creates a random in-memory session and an HttpOnly, SameSite=Lax cookie; HTTPS production mode adds Secure. /me returns the public user, /logout expires the cookie.

This is a local course demo, not a hardened production identity service: sessions disappear on restart and there is no recovery, MFA, durable session database or deployment review.

32. Route permissions at a glance

Auth routes: options and me are read-only; register and login are rate-limited; logout clears a session. The clinic router applies authentication before routes.

- GET /api/dashboard signed-in user; metrics and visits are role-scoped
- GET /api/patients signed-in user; directory/search
- POST /api/patients administrator
- GET /api/veterinarians signed-in user; account-linked roster
- GET /api/appointments administrator all; veterinarian own only
- POST /api/appointments administrator only
- PATCH /api/appointments/:id assigned vet confirms; admin manages
- GET /api/invoices administrator only
- PATCH /api/invoices/:id administrator only

requireRole() is middleware for broad role restrictions. Appointment ownership is checked inside the transition route because the allowed action depends on the record's assigned vet.

33. API validation helpers and dashboard

web/routes/api.js loads the fictional arrays, persistence function and auth guards. Constants define ISO date/time shapes, reason length, visit duration and active statuses. isRealDate rebuilds a local calendar date and reads it back to reject overflow dates. isRealTime verifies hours, minutes and five-minute increments. isWithinClinicHours checks weekday/opening bounds. bySchedule sorts ISO date/time strings; nextAppointmentId selects the next integer ID.

GET /dashboard uses the current role and linked roster to choose records, then calculates today's count, pending confirmation count, patient count and administrator-only invoice totals. Its upcoming records are active, future and sorted. The clinic details accompany the response.

34. API patient and veterinarian endpoints

GET /patients optionally searches all patient fields case-insensitively and returns matching records. POST /patients is administrator-only: it trims and length-checks required fields, assigns an ID, sets Active and a null last-visit date, pushes the record, and persists. If file storage fails, it removes the in-memory addition and propagates the error.

GET /veterinarians returns the auth module's account-backed roster. It does not expose password material. The profile ID returned in this roster is the ID that booking submits and the access control checks compare.

These handlers use the same array objects that clinic.js hydrates. Persistence is local JSON, not a call to the Oracle schema.

35. API appointment creation

POST /appointments first requires administrator role. It resolves a patient and linked veterinarian by numeric ID, trims date/time/reason, checks missing/invalid values, ensures the date is not past, enforces opening days/hours and future time for today, and caps reason length.

A visit consumes 30 minutes. For the selected date, active appointment times are compared to the new start; a conflict less than 30 minutes apart for the same vet returns HTTP 409. The same overlap check for patient prevents double booking an animal with another veterinarian.

On success, server code builds one appointment object with stable IDs/names and status Pending confirmation. It appends, persists, rolls back the array append if persistence throws, then returns HTTP 201 JSON. Status starts pending; the doctor still has to accept it.

36. API appointment transition rules

PATCH /appointments/:id finds one appointment and reads the requested status. A veterinarian must match the appointment's veterinarian ID before any action is considered; another doctor's record is reported as not found. A matching veterinarian may only confirm a Pending confirmation visit. Reconfirmation returns a conflict.

Administrators may cancel any active appointment. They may complete Confirmed or legacy Scheduled visits only after the appointment start time. They cannot complete a pending request. Completed/cancelled appointments cannot be updated again.

Before writing, the handler remembers the previous status. On persistence failure it restores that value and rethrows. A successful update returns the record with its canonical status.

37. API invoice handling and error boundaries

GET /invoices and PATCH /invoices/:id require administrator role. The list optionally filters by status and sorts newest first. Mark paid accepts only a Paid target and rejects duplicate payment marking. The handler saves the original status and restores it if persistence fails.

Unknown API paths return JSON 404 from server.js. Invalid JSON also returns JSON 400. The browser request helper converts error JSON into a user-readable message; authorization errors are not treated as successful empty data.

The API is deliberately smaller than the Oracle project: it has patients, vets, appointments, invoices and account flows, not owner management, pharmacy inventory, prescriptions or Oracle queries.

38. Web demo data and atomic persistence

web/data/clinic.js defines calendar helpers (startOfToday, dateFromToday, toIsoDate), clinic facts, eleven veterinarian profiles, fictional patient and appointment arrays and invoice examples. Dates for initial appointments are relative to the local day so demo records remain upcoming after the calendar changes.

The final block resolves CLINIC_STORE_PATH or the ignored default file, groups persisted collections, and hydrates existing arrays. A malformed store or missing collection throws an explicit error instead of quietly starting from incomplete data. persistClinicData() creates its directory, writes a unique temp file with restrictive mode, renames it over the store, then sets file permissions. The module exports shared arrays/helpers.

39. Test harness and isolation

web/test/api.test.js uses Node's built-in node:test and strict assertions. Before importing the server, it creates unique temporary account/store paths and random invitation/password test values in environment variables. It derives hashes for seeded veterinarian accounts, then writes the temporary store. This prevents tests from touching a developer's ignored local data.

Before/after hooks start Express on an ephemeral port and close connections/remove only their own temporary files. Helper functions perform GET, POST and PATCH requests, extract cookies and construct local calendar dates.

The test suite never uses the currently running website server or Oracle. Deterministic isolated fixtures make authorization and persistence behavior repeatable.

40. API tests and DOM checker

The thirteen API tests cover static page/assets, invitation-gated registration and login, role/endpoint permissions, a long-range Maya appointment and assigned-only confirmation, dashboard data, search/status filters, patient insertion, booking/conflicts, scheduling all account-backed vets, date/time validation, appointment transitions, invoice payment and malformed/unknown API requests.

The Maya regression test books more than seven days ahead, signs into Maya's seeded account, checks the server schedule/dashboard include that pending record, checks a different vet cannot see it, and confirms it using Maya's session.

web/check_dom.js compares JavaScript querySelector and querySelectorAll references against IDs/classes present in public/index.html; a missing selector exits nonzero.

41. CSS: visual architecture

web/public/styles.css is plain CSS, nearly one thousand lines, organized into layers. The first section defines light/base design tokens and component structure. The later dark Liquid Glass override changes tokens, colors, translucent surfaces, shadows, focus treatment and statuses. The final rule additions style worklist metadata, confirmation statuses and agenda elements.

Component blocks cover shell/sidebar/navigation, profile, top bar, summary cards, dashboard grid, schedule, clinic facts, data tables, status pills, search/select controls, dialogs, auth forms, errors/toasts and action buttons.

CSS contains no API logic. Colors and hidden classes improve presentation, not authorization.

42. CSS: layouts, responsive design and accessibility

The sidebar and content flex settings prevent the fixed navigation from stretching and keep wide tables inside a scrollable region. The dashboard uses grid layouts; the seven-day bar uses CSS custom properties for relative activity. Tables use sticky headings, hover/focus states and numeric alignment.

Media queries adapt navigation, content padding, cards, forms, dialogs and tables to tablet and phone widths. A reduced-motion media query limits animation for users who request less motion. Focus-visible outlines remain obvious against the dark theme.

CSS classes such as .admin-only[hidden] respect the HTML hidden state. This is a UX convenience: the route's server-side check must still reject unauthorized actions.

43. Package files and local environment

web/package.json names the CommonJS package and defines start, dev and test scripts. Runtime dependencies are Express, dotenv and node-oracledb. The current web code does not import node-oracledb; it remains an installed declared dependency for a possible future integration.

web/package-lock.json records exact dependency resolutions and integrity metadata so npm install is reproducible. Do not edit the lock file by hand; npm updates it when package dependencies change.

web/.env.example documents PORT and the rule for future database config. web/.gitignore keeps node_modules and real .env out of commits. Root README covers Oracle and website installation separately.

44. Project documentation files

docs/requirements.md is the acceptance specification: tables, sample-data counts, query features, views, package, triggers, security, tests and required file names.

docs/design.md describes every Oracle column, constraint, relationship and index. docs/erd.md contains the Mermaid relationship diagram. docs/normalization.md explains 1NF/2NF/3NF and price snapshots. docs/checklist.md tracks requirements and resolved questions. docs/decisions.md preserves design/debug decisions, including role boundaries and the long-range doctor worklist.

docs/backup.md explains Oracle Data Pump's server-side DIRECTORY and export/import safety. web/README.md is the web-specific user/developer run guide.

45. Root guides and project rules

README.md is the top-level overview and entry point to Oracle rebuild and optional website commands. It explicitly says the web demo is a separate local JSON store, not connected to Oracle.

AGENTS.md holds project-specific instructions: Oracle Free SQL vocabulary, file locations and naming, fresh-schema rebuilds, fake data, no passwords in files and project-folder boundaries. These guide contributors; they are not executed by the web server or Oracle.

The root .gitignore excludes local account hashes, mutable clinic records, env files, temporary atomic writes, database dumps and logs. .DS_Store is OS metadata and has no runtime role.

46. PDF sources and generation

docs/user-guide.md is the printable step-by-step staff guide. docs/technical-handover.md is the source for this extended explanation. The PDFs are generated deliverables in docs/. docs/build_pdf_guides.js formats each marked page, checks page limits, converts UTF-8 Markdown to plain printable text, inserts hard page breaks and calls macOS cupsfilter to create a PDF.

The generator has no runtime dependency in the clinic website and does not alter source code, account hashes or the clinic store. It needs Node.js and the macOS CUPS cupsfilter command. Rebuild both PDFs from the repository root with node docs/build_pdf_guides.js.

The guide source files are kept with the PDFs so a teacher can search, inspect and regenerate the text instead of treating the PDFs as opaque binaries.

47. Safe Oracle runbook

1. Install/open Oracle Database Free and SQLcl.
2. Connect as the schema owner in SQLcl; credentials are entered by the operator, never read from this repository.
3. Check the target schema and back up any data that must be retained.
4. From the repository root, run @sql/run_all.sql.
5. Watch for SQL errors and review DBMS_OUTPUT for each PASS/FAIL.
6. Confirm the final completion prompt and query views/tables if required for the course.

The first reset script drops named project objects. Running the entry point over an existing project schema replaces the sample setup. It is not safe to run casually against a schema with unrelated objects/data. CREATE ROLE may be skipped if privileges are unavailable; a VET schema owner cannot own a role named vet.

48. Safe web runbook and troubleshooting

1. From the project root, run cd web, npm install, then npm start.
2. Keep the server terminal open and browse to http://localhost:3000.
3. Sign in using credentials supplied separately by the local project owner.
4. For code changes, use npm test and node check_dom.js from web/.
5. Stop the process with Ctrl+C.

If login fails, verify account email/password and the local ignored account file. If APIs return 401, sign in again. If changes vanish, check write permissions and the local clinic JSON path. If the page shows stale markup, reload after confirming the server serves the current web/ folder. Do not delete local stores as a debugging shortcut; back them up first.

49. Scope, limitations and honest claims

The web site is a local demo, not an internet deployment. Sessions are in memory, there is no durable database, reset/recovery workflow, e-mail delivery, customer-facing portal, payment processing, clinical-note editor, prescription workflow, vaccination editor or concurrent multi-server coordination.

The Oracle app and web app have different data, identifiers, statuses and business rules. The SQL appointment date column is DATE; web appointments use separate ISO date and time strings. Oracle status values are uppercase SCHEDULED/COMPLETED/CANCELLED; the web introduces Pending confirmation and Confirmed. These representations must be reconciled before integration.

This handover describes verified code, not a guarantee that every production risk has been addressed. Do not enter real personal or clinical information.

50. End-to-end appointment data flow

Administrator browser: selects patient, linked vet, local date/time and reason. main.js POSTs a JSON object to Express. The route authenticates the cookie, requires admin, validates the values, checks active overlaps, creates Pending confirmation, persists it and returns JSON.

Veterinarian browser: reloads data. The API uses the session's linked veterinarian ID to return only that vet's schedule. The dashboard renders all future assigned visits, even beyond the next week. The doctor chooses Confirm visit. The PATCH route compares the appointment's assigned vet ID to the authenticated account, checks Pending confirmation, changes status, persists, and returns the updated record.

The browser then fetches fresh dashboard and appointment data, which removes the pending action and displays Confirmed. Admins can see the same new state on their next refresh.

51. Questions the student should be ready to answer

* Why are bridge tables needed? Because one visit can contain many services/medicines and each service/medicine may occur in many visits.
* Why snapshot prices? Historical billed prices must survive a catalogue price change.
* Why does a vet ID come from the session? A browser-supplied ID is not trusted for authorization.
* Why check permission in the API as well as hide a button? Users can construct direct requests.
* Why aggregate before joining invoices? Joining multiple one-to-many line tables first can multiply rows and inflate totals.
* Why isn't the browser connected to Oracle? The web demo has not yet implemented a database adapter; it uses local fictional JSON.
* Why can roles be skipped? CREATE ROLE requires a database-level privilege, and Oracle reserves the name when the schema owner is already VET.

52. Final handover checklist

Before presenting: open the user guide, demonstrate administrator sign-in and booking, show the pending record on the assigned doctor's account, confirm it, and show that another veterinarian cannot access it. Show the administrator schedule and patient/billing views only if requested.

For the database: explain the 14-table model, runbook safety, query/view examples, package procedures, triggers, least-privilege caveat and test output. Do not claim the browser is wired to Oracle.

Deliver docs/user-guide.pdf to site users and docs/technical-handover.pdf to the teacher. Keep the source Markdown and generator alongside them. Keep local account/store files private. Check Git status before handover; do not add .env, staff-accounts.json, clinic-store.json, or temporary test files.

Complete file inventory - root and docs

The walkthrough covers all project-owned files. Root: .gitignore ignores local state, AGENTS.md is contributor rules, README.md is the project entry, and .DS_Store is inert macOS metadata.

Documentation: docs/requirements.md, checklist.md, design.md, erd.md, normalization.md, decisions.md, and backup.md explain scope, acceptance, schema, relationships, normalization, design history, and Data Pump. user-guide.md and technical-handover.md are editable PDF sources. build_pdf_guides.js creates the printable PDFs. The two .pdf files are the handover artifacts, not application code.

Complete file inventory - SQL

sql/run_all.sql is the entry point. 00_drop_all.sql safely removes named project objects. 01_ddl.sql creates tables, constraints and indexes. 02_data.sql seeds fictional records. 03_queries.sql contains the eighteen business queries. 04_views.sql creates four reporting views. 05_package.sql defines the package contract/body and examples. 06_triggers.sql creates audit, stock and invoice-date triggers and trigger checks. 07_security.sql demonstrates roles, grants and transaction controls. 08_tests.sql runs the final database test suite.

They are separate executable files in one dependency order; @@ includes are relative to the SQLcl script. All are Oracle SQL/PLSQL, not scripts for the web server.

Complete file inventory - web server and tests

web/package.json declares scripts/dependencies; package-lock.json locks their exact graph. .env.example is safe configuration guidance; .gitignore excludes installed dependencies and private environment settings. README.md explains local developer operation.

server.js configures HTTP middleware and routing. routes/auth.js handles identity and roles. routes/api.js handles clinic API operations. data/clinic.js defines/hydrates/persists fictional records. test/api.test.js tests the HTTP API in isolated temporary stores. check_dom.js validates UI selector references. None of the ignored local JSON stores is a tracked source file or safe handover attachment.

Complete file inventory - browser, artifacts and closing

web/public/index.html declares the accessible page and forms. web/public/js/main.js loads, mutates and renders browser state through JSON. web/public/styles.css defines layout, dark theme, responsive behavior and interaction states.

docs/user-guide.pdf explains staff operation. docs/technical-handover.pdf explains how the project is built and connected (and clearly notes the database/web separation). docs/user-guide.md, docs/technical-handover.md and docs/build_pdf_guides.js make both documents maintainable.

For current code behavior, the source files and passing tests are authoritative. When a feature changes, update its code/tests, then this documentation and regenerated PDFs before handing over.